<DSA>
algorithm
data_structures

Data Structures & Algorithms

The Complete A to Z Guide

Mohammad Faiz

Year 2026

Copyright Notice

Copyright (c) 2026 Mohammad Faiz. All Rights Reserved.

No Permission to Use: No person or entity is permitted to use, copy, modify, merge, publish, distribute, sublicense, or sell any part of this content for any purpose (commercial or non-commercial).

Personal Use Only: Only the original author Mohammad Faiz has the authority to use this content in any project or application.

Attribution: If you discuss or reference the concepts, you must provide full credit to the original author Mohammad Faiz.

Table of Contents

Right-click the TOC and select "Update Field" to refresh page numbers

Chapter 1: Introduction to DSA.....1

1.1 What is Data Structure?.....1

1.2 What is Algorithm?.....2

Chapter 2: Arrays.....4

2.1 Introduction to Arrays.....4

2.2 Array Operations.....5

Chapter 3: Linked Lists.....10

3.1 Singly Linked List.....10

3.2 Doubly Linked List.....12

Chapter 4: Stacks.....16

4.1 Stack Implementation.....16

4.2 Stack Applications.....18

Chapter 5: Queues.....22

5.1 Types of Queues.....22

5.2 Queue Applications.....24

Chapter 6: Trees.....28

6.1 Binary Trees.....28

6.2 Binary Search Trees.....30

Chapter 7: Heaps.....36

Chapter 8: Graphs.....40

Chapter 9: Hashing.....48

Chapter 10: Sorting Algorithms.....54

Chapter 11: Searching Algorithms.....66

Chapter 12: Dynamic Programming.....72

Chapter 13: Greedy Algorithms.....80

Chapter 14: Backtracking.....86

Chapter 15: Advanced Data Structures.....92

Chapter 16: Interview Preparation.....100

Chapter 1: Introduction to Data Structures and Algorithms

Welcome to the world of Data Structures and Algorithms! This chapter will introduce you to the fundamental concepts that form the backbone of computer science and programming. Whether you are a beginner or looking to strengthen your foundations, this book will guide you through every concept with simple explanations and practical examples.

1.1 What is Data Structure?

A Data Structure is a way of organizing and storing data in a computer so that it can be accessed and used efficiently. Think of it like organizing your room - if you throw everything randomly, it takes forever to find what you need. But if you organize things properly, you can find anything quickly.

Real-Life Example

Imagine a library. Books are not just thrown randomly on shelves. They are organized by categories, authors, and titles. This organization (data structure) helps you find any book quickly. Similarly, in programming, we organize data so computers can find and use it efficiently.

Types of Data Structures

- Linear Data Structures: Arrays, Linked Lists, Stacks, Queues
- Non-Linear Data Structures: Trees, Graphs
- Hash-based: Hash Tables, Hash Maps

1.2 What is Algorithm?

An Algorithm is a step-by-step set of instructions to solve a specific problem. It is like a recipe for cooking - you follow each step in order to get the desired result. In programming, algorithms tell the computer exactly what to do.

Real-Life Example

Making a cup of tea is an algorithm:

0. Boil water
1. Put tea bag in cup
2. Pour hot water
3. Wait for 3 minutes

4. Remove tea bag and enjoy

1.3 Why Learn DSA?

Learning Data Structures and Algorithms is essential because:

- Efficiency: Write faster and more efficient programs
- Problem Solving: Develop logical thinking and problem-solving skills
- Interviews: Most tech companies test DSA knowledge in interviews
- Foundation: Essential for advanced computer science topics

1.4 Time and Space Complexity

When we talk about algorithms, we need to measure how good they are. We use two main measures:

Time Complexity

Time complexity tells us how much time an algorithm takes to run as the input size grows. We use Big O notation to express this.

Common Time Complexities

- O(1) - Constant Time: Same time regardless of input size
- O(log n) - Logarithmic: Very efficient, like binary search
- O(n) - Linear: Time grows proportionally with input
- O(n log n) - Linearithmic: Efficient sorting algorithms
- $O(n^2)$ - Quadratic: Nested loops, slower
- $O(2^n)$ - Exponential: Very slow, avoid if possible

Space Complexity

Space complexity tells us how much memory an algorithm uses. Just like time, we want to use as little memory as possible.

1.5 Practice Questions

Question 1: What is the time complexity of accessing an array element by index?

Answer: O(1) - Constant time. Arrays store elements in contiguous memory, so we can calculate the exact address directly.

Question 2: Why are data structures important?

Answer: Data structures help organize data efficiently, making operations like search, insert, and delete faster and more memory-efficient.

Question 3: What is the difference between time and space complexity?

Answer: Time complexity measures how fast an algorithm runs, while space complexity measures how much memory it uses.

Chapter 2: Arrays

Arrays are the simplest and most fundamental data structure. They store elements of the same type in contiguous memory locations, making them very efficient for certain operations.

2.1 Introduction to Arrays

An array is a collection of items stored at contiguous memory locations. All elements in an array are of the same data type (like all integers or all characters).

Real-Life Example

Think of a row of lockers in a school. Each locker has a number (index), and you can directly go to any locker if you know its number. This is exactly how arrays work!

Array Declaration

```
// C++ Array Declaration
int numbers[5] = {10, 20, 30, 40, 50};

// Java Array Declaration
int[] numbers = {10, 20, 30, 40, 50};

// Python (List, similar to array)
numbers = [10, 20, 30, 40, 50]
```

2.2 Array Operations

1. Accessing Elements

Array elements are accessed using their index. In most programming languages, array indices start at 0.

```
int firstElement = numbers[0];  // Gets 10
int thirdElement = numbers[2];  // Gets 30
```

2. Inserting Elements

Inserting at the end is O(1), but inserting in the middle requires shifting elements, making it O(n).

```
// Insert 25 at index 2
// Shift elements from index 2 onwards
for (int i = 4; i >= 2; i--) {
    numbers[i+1] = numbers[i];
```

```
}
numbers[2] = 25;
```

3. Deleting Elements

Deleting also requires shifting elements to fill the gap, making it O(n) in the worst case.

```
// Delete element at index 2
for (int i = 2; i < 4; i++) {
    numbers[i] = numbers[i+1];
}
```

4. Searching Elements

Linear search checks each element one by one.

```
// Linear Search
for (int i = 0; i < 5; i++) {
    if (numbers[i] == 30) {
        return i;  // Found at index 2
    }
}
return -1;  // Not found
```

2.3 Types of Arrays

1. One-Dimensional Arrays

A simple list of elements, like a single row of lockers.

```
int arr[5] = {1, 2, 3, 4, 5};
```

2. Two-Dimensional Arrays

Like a table with rows and columns, or a matrix in mathematics.

```
int matrix[3][3] = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};
```

3. Multi-Dimensional Arrays

Arrays with more than two dimensions, used for complex data representation.

2.4 Advantages and Disadvantages

Advantages

- Fast access: O(1) time to access any element by index
- Memory efficient: No extra memory for pointers
- Cache friendly: Contiguous memory improves performance

Disadvantages

- Fixed size: Size must be known at declaration (in most languages)
- Expensive insertions/deletions: O(n) for middle operations
- Same data type: Cannot mix different types

2.5 Practice Questions

Question 1: Find the maximum element in an array

```
int findMax(int arr[], int n) {
    int max = arr[0];
    for (int i = 1; i < n; i++) {
        if (arr[i] > max) {
            max = arr[i];
        }
    }
    return max;
}
// Time Complexity: O(n)
```

Question 2: Reverse an array

```
void reverse(int arr[], int n) {
    int start = 0, end = n - 1;
    while (start < end) {
        swap(arr[start], arr[end]);
        start++;
        end--;
    }
}
// Time Complexity: O(n)
```

Question 3: Find if array contains duplicates

```
bool hasDuplicate(int arr[], int n) {
    sort(arr, arr + n);
```

```
    for (int i = 0; i < n - 1; i++) {
        if (arr[i] == arr[i + 1])
            return true;
    }
    return false;
}
// Time Complexity: O(n log n) due to sorting
```

Interview Question: What is the time complexity of array insertion at the beginning?

Answer: O(n) because all existing elements need to be shifted one position to the right.

Chapter 3: Linked Lists

Linked Lists are linear data structures where elements are stored in nodes, and each node points to the next node. Unlike arrays, elements are not stored in contiguous memory locations.

3.1 Singly Linked List

In a singly linked list, each node contains data and a pointer to the next node. The last node points to NULL.

Node Structure

```
struct Node {
    int data;
    Node* next;
};
```

Real-Life Example

Think of a treasure hunt. Each clue (node) tells you where to find the next clue (next pointer). You follow the chain until you reach the treasure (NULL).

Creating a Node

```
Node* createNode(int data) {
    Node* newNode = new Node();
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}
```

Inserting at the Beginning

```
void insertAtBeginning(Node*& head, int data) {
    Node* newNode = createNode(data);
    newNode->next = head;
    head = newNode;
}
// Time Complexity: O(1)
```

Inserting at the End

```
void insertAtEnd(Node*& head, int data) {
    Node* newNode = createNode(data);
    if (head == NULL) {
        head = newNode;
```

```
        return;
    }
    Node* temp = head;
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = newNode;
}
// Time Complexity: O(n)
```

Deleting a Node

```
void deleteNode(Node*& head, int key) {
    if (head == NULL) return;

    if (head->data == key) {
        Node* temp = head;
        head = head->next;
        delete temp;
        return;
    }

    Node* temp = head;
    while (temp->next != NULL && temp->next->data != key) {
        temp = temp->next;
    }

    if (temp->next != NULL) {
        Node* toDelete = temp->next;
        temp->next = temp->next->next;
        delete toDelete;
    }
}
```

3.2 Doubly Linked List

In a doubly linked list, each node has two pointers - one to the next node and one to the previous node. This allows traversal in both directions.

Node Structure

```
struct DNode {
    int data;
    DNode* next;
    DNode* prev;
};
```

Advantages of Doubly Linked List

- Can traverse in both directions
- Easy to delete a node if we have its pointer
- Can insert before a node easily

3.3 Circular Linked List

In a circular linked list, the last node points back to the first node instead of NULL. This creates a circle of nodes.

Applications

- Round-robin scheduling in operating systems
- Music playlists (repeat mode)
- Multiplayer games (turn-based)

3.4 Linked List vs Array

Aspect	Array	Linked List
Memory	Contiguous	Non-contiguous
Access	O(1)	O(n)
Insertion	O(n)	O(1)
Deletion	O(n)	O(1)
Size	Fixed	Dynamic

3.5 Practice Questions

Question 1: Reverse a Linked List

```
Node* reverse(Node* head) {
    Node* prev = NULL;
    Node* current = head;
    Node* next = NULL;

    while (current != NULL) {
        next = current->next;
```

```
        current->next = prev;
        prev = current;
        current = next;
    }
    return prev;
}
// Time Complexity: O(n)
```

Question 2: Detect Loop in Linked List (Floyd's Cycle)

```
bool hasCycle(Node* head) {
    if (head == NULL) return false;

    Node* slow = head;
    Node* fast = head;

    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;

        if (slow == fast)
            return true;
    }
    return false;
}
// Time Complexity: O(n), Space: O(1)
```

Interview Question: Why use linked lists over arrays?

Answer: Linked lists are better when you need frequent insertions/deletions, especially at the beginning or middle, as they don't require shifting elements like arrays do.

Chapter 4: Stacks

A Stack is a linear data structure that follows the Last-In-First-Out (LIFO) principle. The last element added is the first one to be removed.

4.1 Stack Implementation

Real-Life Example

Think of a stack of plates. You can only add a new plate on top, and you can only remove the top plate. You cannot take a plate from the middle without removing the plates above it first.

Stack Operations

- Push: Add an element to the top of the stack
- Pop: Remove and return the top element
- Peek/Top: View the top element without removing it
- isEmpty: Check if the stack is empty
- Size: Get the number of elements in the stack

Array-based Implementation

```
class Stack {
    int top;
    int capacity;
    int* array;

public:
    Stack(int size) {
        capacity = size;
        array = new int[capacity];
        top = -1;
    }

    void push(int x) {
        if (top >= capacity - 1) {
            cout << "Stack Overflow";
            return;
        }
        array[++top] = x;
    }

    int pop() {
```

```
    if (top < 0) {
        cout << "Stack Underflow";
        return -1;
    }
    return array[top--];
}

int peek() {
    if (top < 0) return -1;
    return array[top];
}

bool isEmpty() {
    return top < 0;
}
};
```

Linked List-based Implementation

```
class StackLL {
    Node* top;

public:
    StackLL() { top = NULL; }

    void push(int x) {
        Node* newNode = new Node(x);
        newNode->next = top;
        top = newNode;
    }

    int pop() {
        if (top == NULL) return -1;
        int popped = top->data;
        Node* temp = top;
        top = top->next;
        delete temp;
        return popped;
    }

    int peek() {
        if (top == NULL) return -1;
        return top->data;
    }

    bool isEmpty() {
        return top == NULL;
```

```
    }
};
```

4.2 Stack Applications

1. Expression Evaluation

Stacks are used to evaluate mathematical expressions, especially converting infix to postfix and then evaluating.

2. Function Call Management

When functions call other functions, the system uses a call stack to keep track of where to return after each function completes.

3. Undo Operations

Text editors use stacks to implement undo functionality. Each action is pushed onto the stack, and undo pops the last action.

4. Browser History

Browsers use stacks to implement the back button. Each visited page is pushed, and clicking back pops the stack.

5. Balanced Parentheses Check

```
bool isBalanced(string expr) {
    Stack s;
    for (char c : expr) {
        if (c == '(' || c == '[' || c == '{')
            s.push(c);
        else if (c == ')' || c == ']' || c == '}') {
            if (s.isEmpty()) return false;
            char top = s.pop();
            if ((c == ')' && top != '(') ||
                (c == ']' && top != '[') ||
                (c == '}' && top != '{'))
                return false;
        }
    }
    return s.isEmpty();
}
```

4.3 Practice Questions

Question 1: Reverse a String using Stack

```
string reverseString(string str) {
    Stack s;
    for (char c : str)
        s.push(c);

    string reversed = "";
    while (!s.isEmpty())
        reversed += s.pop();

    return reversed;
}
// Time Complexity: O(n)
```

Question 2: Next Greater Element

```
vector<int> nextGreater(vector<int>& arr) {
    int n = arr.size();
    vector<int> result(n, -1);
    stack<int> s;

    for (int i = 0; i < n; i++) {
        while (!s.empty() && arr[s.top()] < arr[i]) {
            result[s.top()] = arr[i];
            s.pop();
        }
        s.push(i);
    }
    return result;
}
```

Interview Question: What is stack overflow?

Answer: Stack overflow occurs when we try to push an element onto a full stack (in array implementation) or when the recursion depth exceeds the system stack limit.

Chapter 5: Queues

A Queue is a linear data structure that follows the First-In-First-Out (FIFO) principle. The first element added is the first one to be removed.

5.1 Types of Queues

Real-Life Example

Think of a line at a ticket counter. The person who arrives first gets served first. This is exactly how queues work in programming.

1. Simple Queue

Basic queue with enqueue (add to rear) and dequeue (remove from front) operations.

```
class Queue {
    int front, rear, size;
    int capacity;
    int* array;

public:
    Queue(int cap) {
        capacity = cap;
        front = size = 0;
        rear = capacity - 1;
        array = new int[capacity];
    }

    void enqueue(int item) {
        if (isFull()) return;
        rear = (rear + 1) % capacity;
        array[rear] = item;
        size++;
    }

    int dequeue() {
        if (isEmpty()) return -1;
        int item = array[front];
        front = (front + 1) % capacity;
        size--;
        return item;
    }

    bool isFull() { return size == capacity; }
```

```
    bool isEmpty() { return size == 0; }
};
```

2. Circular Queue

In a circular queue, the last position is connected back to the first position, forming a circle. This efficiently uses space.

3. Priority Queue

Elements are served based on priority, not just arrival order. Higher priority elements are dequeued before lower priority ones.

4. Deque (Double-Ended Queue)

Elements can be added or removed from both ends. It combines features of both stack and queue.

5.2 Queue Applications

- CPU Scheduling: Processes wait in a queue for CPU time
- Printer Spooling: Print jobs are queued and processed in order
- Breadth-First Search (BFS): Used in graph traversal
- Handling requests on a single shared resource
- Asynchronous data transfer between processes

5.3 Implementing Queue using Two Stacks

```
class QueueUsingStacks {
    stack<int> s1, s2;

public:
    void enqueue(int x) {
        s1.push(x);
    }

    int dequeue() {
        if (s2.empty()) {
            if (s1.empty()) return -1;
            while (!s1.empty()) {
                s2.push(s1.top());
                s1.pop();
            }
        }
        int x = s2.top();
```

```
        s2.pop();
        return x;
    }
};
```

5.4 Practice Questions

Question 1: Reverse First K Elements of Queue

```
void reverseFirstK(queue<int>& q, int k) {
    if (q.empty() || k > q.size()) return;

    stack<int> s;
    for (int i = 0; i < k; i++) {
        s.push(q.front());
        q.pop();
    }

    while (!s.empty()) {
        q.push(s.top());
        s.pop();
    }

    for (int i = 0; i < q.size() - k; i++) {
        q.push(q.front());
        q.pop();
    }
}
```

Question 2: Generate Binary Numbers from 1 to N

```
vector<string> generateBinary(int n) {
    vector<string> result;
    queue<string> q;
    q.push("1");

    for (int i = 0; i < n; i++) {
        string curr = q.front();
        q.pop();
        result.push_back(curr);
        q.push(curr + "0");
        q.push(curr + "1");
    }
    return result;
}
```

Interview Question: Difference between Stack and Queue?

Answer: Stack follows LIFO (Last-In-First-Out) where the last element added is removed first. Queue follows FIFO (First-In-First-Out) where the first element added is removed first.

Chapter 6: Trees

A Tree is a hierarchical data structure consisting of nodes connected by edges. It starts with a root node and each node can have child nodes.

6.1 Binary Trees

A Binary Tree is a tree where each node has at most two children, referred to as left child and right child.

Node Structure

```
struct TreeNode {
    int data;
    TreeNode* left;
    TreeNode* right;

    TreeNode(int val) {
        data = val;
        left = right = NULL;
    }
};
```

Tree Traversals

Tree traversal means visiting all nodes of a tree in a specific order.

1. Inorder Traversal (Left-Root-Right)

```
void inorder(TreeNode* root) {
    if (root == NULL) return;
    inorder(root->left);
    cout << root->data << " ";
    inorder(root->right);
}
```

2. Preorder Traversal (Root-Left-Right)

```
void preorder(TreeNode* root) {
    if (root == NULL) return;
    cout << root->data << " ";
    preorder(root->left);
    preorder(root->right);
}
```

3. Postorder Traversal (Left-Right-Root)

```
void postorder(TreeNode* root) {
    if (root == NULL) return;
    postorder(root->left);
    postorder(root->right);
    cout << root->data << " ";
}
```

4. Level Order Traversal (BFS)

```
void levelOrder(TreeNode* root) {
    if (root == NULL) return;

    queue<TreeNode*> q;
    q.push(root);

    while (!q.empty()) {
        TreeNode* curr = q.front();
        q.pop();
        cout << curr->data << " ";

        if (curr->left) q.push(curr->left);
        if (curr->right) q.push(curr->right);
    }
}
```

6.2 Binary Search Trees (BST)

A Binary Search Tree is a binary tree where for each node:

- All values in the left subtree are smaller than the node's value
- All values in the right subtree are greater than the node's value
- Both left and right subtrees are also BSTs

BST Operations

Search in BST

```
TreeNode* search(TreeNode* root, int key) {
    if (root == NULL || root->data == key)
        return root;

    if (key < root->data)
        return search(root->left, key);
```

```
    return search(root->right, key);
}
// Time Complexity: O(h) where h is height
```

Insert in BST

```
TreeNode* insert(TreeNode* root, int key) {
    if (root == NULL)
        return new TreeNode(key);

    if (key < root->data)
        root->left = insert(root->left, key);
    else if (key > root->data)
        root->right = insert(root->right, key);

    return root;
}
```

6.3 Types of Binary Trees

1. Full Binary Tree

Every node has either 0 or 2 children. No node has exactly 1 child.

2. Complete Binary Tree

All levels are completely filled except possibly the last level, which is filled from left to right.

3. Perfect Binary Tree

All internal nodes have two children and all leaves are at the same level.

4. Balanced Binary Tree

The height of the tree is O(log n). AVL trees and Red-Black trees are self-balancing BSTs.

6.4 AVL Trees

AVL Tree is a self-balancing BST where the difference between heights of left and right subtrees (balance factor) cannot be more than 1 for all nodes.

Rotations in AVL Tree

- Left Rotation
- Right Rotation

- Left-Right Rotation
- Right-Left Rotation

6.5 Practice Questions

Question 1: Find Height of Binary Tree

```
int height(TreeNode* root) {
    if (root == NULL) return -1;
    return 1 + max(height(root->left), height(root->right));
}
```

Question 2: Check if Two Trees are Identical

```
bool isSameTree(TreeNode* p, TreeNode* q) {
    if (p == NULL && q == NULL) return true;
    if (p == NULL || q == NULL) return false;

    return (p->data == q->data) &&
           isSameTree(p->left, q->left) &&
           isSameTree(p->right, q->right);
}
```

Question 3: Validate BST

```
bool isValidBST(TreeNode* root, long minVal, long maxVal) {
    if (root == NULL) return true;

    if (root->data <= minVal || root->data >= maxVal)
        return false;

    return isValidBST(root->left, minVal, root->data) &&
           isValidBST(root->right, root->data, maxVal);
}
```

Interview Question: Why use BST over regular binary tree?

Answer: BST provides O(log n) time complexity for search, insert, and delete operations in balanced cases, whereas a regular binary tree may require O(n) time.

Chapter 7: Heaps and Priority Queues

A Heap is a special tree-based data structure that satisfies the heap property. It is commonly used to implement priority queues.

7.1 Types of Heaps

1. Max Heap

In a Max Heap, the value of each parent node is greater than or equal to the values of its children. The root contains the maximum value.

2. Min Heap

In a Min Heap, the value of each parent node is less than or equal to the values of its children. The root contains the minimum value.

7.2 Heap Representation

Heaps are typically implemented using arrays for efficiency. For a node at index i:

- Parent index: (i-1) / 2
- Left child index: 2*i + 1
- Right child index: 2*i + 2

7.3 Heap Operations

Heapify

Heapify is the process of converting a binary tree into a heap structure. It ensures the heap property is maintained.

```
void heapify(int arr[], int n, int i) {
    int largest = i;
    int left = 2*i + 1;
    int right = 2*i + 2;

    if (left < n && arr[left] > arr[largest])
        largest = left;

    if (right < n && arr[right] > arr[largest])
        largest = right;
```

```
    if (largest != i) {
        swap(arr[i], arr[largest]);
        heapify(arr, n, largest);
    }
}
```

Insert into Heap

```
void insert(int arr[], int& n, int key) {
    n++;
    int i = n - 1;
    arr[i] = key;

    // Fix the max heap property
    while (i != 0 && arr[(i-1)/2] < arr[i]) {
        swap(arr[i], arr[(i-1)/2]);
        i = (i-1)/2;
    }
}
```

Extract Max (Delete Root)

```
int extractMax(int arr[], int& n) {
    if (n <= 0) return -1;
    if (n == 1) {
        n--;
        return arr[0];
    }

    int root = arr[0];
    arr[0] = arr[n-1];
    n--;
    heapify(arr, n, 0);

    return root;
}
```

7.4 Applications of Heaps

- Priority Queues: Efficiently get the highest/lowest priority element
- Heap Sort: O(n log n) sorting algorithm
- Graph Algorithms: Dijkstra's and Prim's algorithms
- Finding Kth Largest/Smallest Element
- Median Maintenance in a Stream

7.5 Practice Questions

Question 1: Find Kth Largest Element

```
int findKthLargest(vector<int>& nums, int k) {
    priority_queue<int, vector<int>, greater<int>> minHeap;

    for (int num : nums) {
        minHeap.push(num);
        if (minHeap.size() > k)
            minHeap.pop();
    }

    return minHeap.top();
}
// Time: O(n log k), Space: O(k)
```

Question 2: Merge K Sorted Arrays

```
vector<int> mergeKSorted(vector<vector<int>>& arrays) {
    vector<int> result;
    priority_queue<pair<int, pair<int,int>>, vector<pair<int, pair<int,int>>>, greater<>> minHeap;

    for (int i = 0; i < arrays.size(); i++)
        if (!arrays[i].empty())
            minHeap.push({arrays[i][0], {i, 0}});

    while (!minHeap.empty()) {
        auto curr = minHeap.top(); minHeap.pop();
        result.push_back(curr.first);
        int arrIdx = curr.second.first;
        int elemIdx = curr.second.second;

        if (elemIdx + 1 < arrays[arrIdx].size())
            minHeap.push({arrays[arrIdx][elemIdx+1], {arrIdx, elemIdx+1}});
    }
    return result;
}
```

Interview Question: When would you use a heap over a sorted array?

Answer: Use a heap when you need to frequently insert elements and extract min/max. Heap operations are O(log n) while maintaining sorted order in an array would be O(n) for insertions.

Chapter 8: Graphs

A Graph is a non-linear data structure consisting of vertices (nodes) connected by edges. Graphs are used to represent networks, relationships, and connections.

8.1 Graph Terminology

- Vertex (Node): A fundamental unit of a graph
- Edge: A connection between two vertices
- Directed Graph: Edges have direction (one-way)
- Undirected Graph: Edges have no direction (two-way)
- Weighted Graph: Edges have associated weights/costs
- Degree: Number of edges connected to a vertex
- Path: A sequence of vertices connected by edges
- Cycle: A path that starts and ends at the same vertex

8.2 Graph Representation

1. Adjacency Matrix

A 2D array where matrix[i][j] = 1 if there is an edge from vertex i to vertex j.

```
// Adjacency Matrix for undirected graph
int adjMatrix[V][V] = {0};

// Add edge between vertices u and v
void addEdge(int u, int v) {
    adjMatrix[u][v] = 1;
    adjMatrix[v][u] = 1;  // For undirected
}

// Space: O(V²)
```

2. Adjacency List

An array of lists where each list contains neighbors of a vertex.

```
// Adjacency List
vector<int> adjList[V];

void addEdge(int u, int v) {
```

```
    adjList[u].push_back(v);
    adjList[v].push_back(u);  // For undirected
}

// Space: O(V + E)
```

8.3 Graph Traversals

1. Breadth-First Search (BFS)

BFS explores all neighbors at the current depth before moving to the next level. It uses a queue.

```
void BFS(int start, vector<int> adj[], int V) {
    vector<bool> visited(V, false);
    queue<int> q;

    visited[start] = true;
    q.push(start);

    while (!q.empty()) {
        int u = q.front();
        q.pop();
        cout << u << " ";

        for (int v : adj[u]) {
            if (!visited[v]) {
                visited[v] = true;
                q.push(v);
            }
        }
    }
}
// Time: O(V + E), Space: O(V)
```

2. Depth-First Search (DFS)

DFS explores as far as possible along each branch before backtracking. It uses recursion or a stack.

```
void DFS(int u, vector<int> adj[], vector<bool>& visited) {
    visited[u] = true;
    cout << u << " ";

    for (int v : adj[u]) {
        if (!visited[v])
            DFS(v, adj, visited);
    }
}
```

```
void DFSUtil(int V, vector<int> adj[]) {
    vector<bool> visited(V, false);
    for (int i = 0; i < V; i++)
        if (!visited[i])
            DFS(i, adj, visited);
}
// Time: O(V + E), Space: O(V)
```

8.4 Shortest Path Algorithms

Dijkstra's Algorithm

Finds the shortest path from a source vertex to all other vertices in a weighted graph with non-negative weights.

```
vector<int> dijkstra(int V, vector<pair<int,int>> adj[], int src) {
    vector<int> dist(V, INT_MAX);
    dist[src] = 0;

    priority_queue<pair<int,int>, vector<pair<int,int>>, greater<>> pq;
    pq.push({0, src});

    while (!pq.empty()) {
        int d = pq.top().first;
        int u = pq.top().second;
        pq.pop();

        if (d > dist[u]) continue;

        for (auto& edge : adj[u]) {
            int v = edge.first;
            int w = edge.second;

            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                pq.push({dist[v], v});
            }
        }
    }
    return dist;
}
// Time: O((V + E) log V)
```

Bellman-Ford Algorithm

Finds shortest paths from a source vertex to all other vertices. Can handle negative weights but not

negative cycles.

```
vector<int> bellmanFord(int V, vector<vector<int>>& edges, int src) {
    vector<int> dist(V, INT_MAX);
    dist[src] = 0;

    // Relax all edges V-1 times
    for (int i = 0; i < V - 1; i++) {
        for (auto& e : edges) {
            int u = e[0], v = e[1], w = e[2];
            if (dist[u] != INT_MAX && dist[u] + w < dist[v])
                dist[v] = dist[u] + w;
        }
    }

    // Check for negative cycles
    for (auto& e : edges) {
        int u = e[0], v = e[1], w = e[2];
        if (dist[u] != INT_MAX && dist[u] + w < dist[v])
            return {};  // Negative cycle exists
    }
    return dist;
}
// Time: O(V * E)
```

8.5 Minimum Spanning Tree

Kruskal's Algorithm

Builds MST by adding edges in increasing order of weight, skipping edges that would form a cycle.

Prim's Algorithm

Builds MST by starting from a vertex and always adding the minimum weight edge that connects a visited vertex to an unvisited vertex.

8.6 Practice Questions

Question 1: Detect Cycle in Undirected Graph

```
bool hasCycle(int V, vector<int> adj[]) {
    vector<bool> visited(V, false);

    for (int i = 0; i < V; i++) {
        if (!visited[i])
            if (dfsCycle(i, -1, adj, visited))
```

```
            return true;
    }
    return false;
}

bool dfsCycle(int u, int parent, vector<int> adj[], vector<bool>& visited) {
    visited[u] = true;
    for (int v : adj[u]) {
        if (!visited[v]) {
            if (dfsCycle(v, u, adj, visited))
                return true;
        } else if (v != parent)
            return true;
    }
    return false;
}
```

Question 2: Topological Sort

```
vector<int> topologicalSort(int V, vector<int> adj[]) {
    vector<int> inDegree(V, 0);
    for (int i = 0; i < V; i++)
        for (int v : adj[i])
            inDegree[v]++;

    queue<int> q;
    for (int i = 0; i < V; i++)
        if (inDegree[i] == 0)
            q.push(i);

    vector<int> result;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        result.push_back(u);
        for (int v : adj[u])
            if (--inDegree[v] == 0)
                q.push(v);
    }
    return result;
}
```

Interview Question: When to use BFS vs DFS?

Answer: Use BFS for finding shortest paths in unweighted graphs and when the solution is near the root. Use DFS for exploring all paths, detecting cycles, and when the solution is deep in the tree.

Chapter 9: Hashing and Hash Tables

Hashing is a technique that converts a large input (key) into a smaller, fixed-size value (hash) using a hash function. Hash tables use this technique for fast data retrieval.

9.1 Hash Function

A hash function maps keys to indices in a hash table. A good hash function should:

- Be fast to compute
- Distribute keys uniformly across the table
- Minimize collisions

Common Hash Functions

```
// Division Method
int hash(int key, int tableSize) {
    return key % tableSize;
}

// Multiplication Method
int hashMultiplication(int key, int tableSize) {
    double A = 0.6180339887;  // Golden ratio
    return (int)(tableSize * (key * A - (int)(key * A)));
}
```

9.2 Collision Resolution

When two keys hash to the same index, we have a collision. We need methods to handle this.

1. Separate Chaining

Each table entry points to a linked list of elements that hash to that index.

```
class HashTable {
    vector<list<pair<int, string>>> table;
    int size;

    int hash(int key) {
        return key % size;
    }

public:
    HashTable(int s) : size(s) {
```

```
    table.resize(size);
  }

  void insert(int key, string value) {
    int index = hash(key);
    table[index].push_back({key, value});
  }

  string search(int key) {
    int index = hash(key);
    for (auto& p : table[index])
      if (p.first == key)
        return p.second;
    return "Not found";
  }
};
```

2. Open Addressing

All elements are stored in the hash table itself. When collision occurs, we probe for the next empty slot.

Linear Probing

```
// If index is occupied, try (index + 1) % size
int linearProbe(int key, int table[], int size) {
  int index = hash(key, size);
  while (table[index] != EMPTY && table[index] != DELETED) {
    index = (index + 1) % size;
  }
  return index;
}
```

Quadratic Probing

```
// Try (index + i²) % size for i = 0, 1, 2, ...
int quadraticProbe(int key, int table[], int size, int i) {
  return (hash(key, size) + i*i) % size;
}
```

9.3 Applications of Hashing

- Database Indexing: Fast data retrieval
- Caching: Storing frequently accessed data
- Password Verification: Store hash instead of actual password
- Counting Frequencies: Count occurrences of elements

- Finding Duplicates: Detect duplicate elements
- Two Sum Problem: Find pairs that sum to target

9.4 Practice Questions

Question 1: Two Sum Problem

```
vector<int> twoSum(vector<int>& nums, int target) {
    unordered_map<int, int> map;

    for (int i = 0; i < nums.size(); i++) {
        int complement = target - nums[i];
        if (map.find(complement) != map.end())
            return {map[complement], i};
        map[nums[i]] = i;
    }
    return {};
}
// Time: O(n), Space: O(n)
```

Question 2: First Non-Repeating Character

```
int firstUniqChar(string s) {
    unordered_map<char, int> freq;

    for (char c : s)
        freq[c]++;

    for (int i = 0; i < s.length(); i++)
        if (freq[s[i]] == 1)
            return i;

    return -1;
}
// Time: O(n), Space: O(1) - at most 26 characters
```

Question 3: Group Anagrams

```
vector<vector<string>> groupAnagrams(vector<string>& strs) {
    unordered_map<string, vector<string>> groups;

    for (string& s : strs) {
        string key = s;
        sort(key.begin(), key.end());
        groups[key].push_back(s);
    }
```

```
    vector<vector<string>> result;
    for (auto& pair : groups)
        result.push_back(pair.second);

    return result;
}
```

Interview Question: What makes a good hash function?

Answer: A good hash function should be deterministic (same input always gives same output), fast to compute, distribute keys uniformly to minimize collisions, and use all input data.

Chapter 10: Sorting Algorithms

Sorting is arranging data in a particular order (ascending or descending). It is one of the most fundamental operations in computer science.

10.1 Comparison-Based Sorting

1. Bubble Sort

Repeatedly steps through the list, compares adjacent elements and swaps them if they are in the wrong order.

```
void bubbleSort(int arr[], int n) {
    for (int i = 0; i < n-1; i++) {
        for (int j = 0; j < n-i-1; j++) {
            if (arr[j] > arr[j+1])
                swap(arr[j], arr[j+1]);
        }
    }
}
// Time: O(n²), Space: O(1)
```

2. Selection Sort

Finds the minimum element and places it at the beginning, then repeats for the remaining elements.

```
void selectionSort(int arr[], int n) {
    for (int i = 0; i < n-1; i++) {
        int minIdx = i;
        for (int j = i+1; j < n; j++)
            if (arr[j] < arr[minIdx])
                minIdx = j;
        swap(arr[i], arr[minIdx]);
    }
}
// Time: O(n²), Space: O(1)
```

3. Insertion Sort

Builds the final sorted array one item at a time, inserting each element into its proper position.

```
void insertionSort(int arr[], int n) {
    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key) {
```

```
            arr[j+1] = arr[j];
            j--;
        }
        arr[j+1] = key;
    }
}
// Time: O(n²), Space: O(1)
// Best case: O(n) when array is already sorted
```

4. Merge Sort

Divide and conquer algorithm that divides the array into halves, sorts them, and merges them back.

```
void merge(int arr[], int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;
    int L[n1], R[n2];

    for (int i = 0; i < n1; i++) L[i] = arr[left + i];
    for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) arr[k++] = L[i++];
        else arr[k++] = R[j++];
    }
    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];
}

void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}
// Time: O(n log n), Space: O(n)
```

5. Quick Sort

Picks a pivot element and partitions the array around it, then recursively sorts the subarrays.

```
int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low - 1;
```

```
    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(arr[i], arr[j]);
        }
    }
    swap(arr[i+1], arr[high]);
    return i + 1;
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}
// Time: O(n log n) average, O(n²) worst
// Space: O(log n)
```

6. Heap Sort

Uses a heap data structure to sort elements.

```
void heapSort(int arr[], int n) {
    // Build max heap
    for (int i = n/2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    // Extract elements one by one
    for (int i = n-1; i > 0; i--) {
        swap(arr[0], arr[i]);
        heapify(arr, i, 0);
    }
}
// Time: O(n log n), Space: O(1)
```

10.2 Non-Comparison Sorting

1. Counting Sort

Counts occurrences of each element and calculates positions. Works for integers in a known range.

```
void countingSort(int arr[], int n, int range) {
    int count[range + 1] = {0};
    int output[n];
```

```
    for (int i = 0; i < n; i++)
        count[arr[i]]++;

    for (int i = 1; i <= range; i++)
        count[i] += count[i-1];

    for (int i = n-1; i >= 0; i--) {
        output[count[arr[i]] - 1] = arr[i];
        count[arr[i]]--;
    }

    for (int i = 0; i < n; i++)
        arr[i] = output[i];
}
// Time: O(n + k), Space: O(n + k) where k is range
```

2. Radix Sort

Sorts numbers digit by digit from least significant to most significant.

```
void radixSort(int arr[], int n) {
    int maxVal = *max_element(arr, arr + n);

    for (int exp = 1; maxVal / exp > 0; exp *= 10)
        countingSortByDigit(arr, n, exp);
}
// Time: O(d * (n + k)) where d is number of digits
```

10.3 Sorting Algorithm Comparison

Algorithm	Best	Average	Worst	Space
Bubble Sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(1)
Insertion Sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)
Quick Sort	O(n log n)	O(n log n)	$O(n^2)$	O(log n)
Heap Sort	O(n log n)	O(n log n)	O(n log n)	O(1)

10.4 Practice Questions

Question 1: Sort an array of 0s, 1s, and 2s (Dutch National Flag)

```
void sortColors(vector<int>& nums) {
    int low = 0, mid = 0, high = nums.size() - 1;

    while (mid <= high) {
        if (nums[mid] == 0) {
            swap(nums[low++], nums[mid++]);
        } else if (nums[mid] == 1) {
            mid++;
        } else {
            swap(nums[mid], nums[high--]);
        }
    }
}
// Time: O(n), Space: O(1)
```

Question 2: Merge Intervals

```
vector<vector<int>> merge(vector<vector<int>>& intervals) {
    if (intervals.empty()) return {};

    sort(intervals.begin(), intervals.end());

    vector<vector<int>> result;
    result.push_back(intervals[0]);

    for (int i = 1; i < intervals.size(); i++) {
        if (intervals[i][0] <= result.back()[1]) {
            result.back()[1] = max(result.back()[1], intervals[i][1]);
        } else {
            result.push_back(intervals[i]);
        }
    }
    return result;
}
```

Interview Question: When would you use QuickSort over MergeSort?

Answer: Use QuickSort when memory is limited (O(log n) space vs O(n) for MergeSort) and when average-case performance matters. However, MergeSort is preferred for linked lists and when stable sorting is needed.

Chapter 11: Searching Algorithms

Searching is the process of finding a specific element in a data structure. Efficient searching is crucial for many applications.

11.1 Linear Search

Linear search checks each element one by one until the target is found or the end is reached.

```
int linearSearch(int arr[], int n, int target) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == target)
            return i;
    }
    return -1;
}
// Time: O(n), Space: O(1)
```

11.2 Binary Search

Binary search works on sorted arrays by repeatedly dividing the search interval in half.

```
int binarySearch(int arr[], int n, int target) {
    int left = 0, right = n - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (arr[mid] == target)
            return mid;

        if (arr[mid] < target)
            left = mid + 1;
        else
            right = mid - 1;
    }
    return -1;
}
// Time: O(log n), Space: O(1)
```

Recursive Binary Search

```
int binarySearchRecursive(int arr[], int left, int right, int target) {
    if (left > right) return -1;
```

```
    int mid = left + (right - left) / 2;

    if (arr[mid] == target) return mid;
    if (arr[mid] > target)
        return binarySearchRecursive(arr, left, mid - 1, target);
    return binarySearchRecursive(arr, mid + 1, right, target);
}
```

11.3 Search in Rotated Sorted Array

```
int searchRotated(vector<int>& nums, int target) {
    int left = 0, right = nums.size() - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (nums[mid] == target) return mid;

        // Left half is sorted
        if (nums[left] <= nums[mid]) {
            if (target >= nums[left] && target < nums[mid])
                right = mid - 1;
            else
                left = mid + 1;
        }
        // Right half is sorted
        else {
            if (target > nums[mid] && target <= nums[right])
                left = mid + 1;
            else
                right = mid - 1;
        }
    }
    return -1;
}
// Time: O(log n)
```

11.4 Finding Bounds

Find First and Last Position

```
vector<int> searchRange(vector<int>& nums, int target) {
    return {findFirst(nums, target), findLast(nums, target)};
}

int findFirst(vector<int>& nums, int target) {
```

```
    int left = 0, right = nums.size() - 1;
    int result = -1;

    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (nums[mid] >= target)
            right = mid - 1;
        else
            left = mid + 1;
        if (nums[mid] == target) result = mid;
    }
    return result;
}

int findLast(vector<int>& nums, int target) {
    int left = 0, right = nums.size() - 1;
    int result = -1;

    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (nums[mid] <= target)
            left = mid + 1;
        else
            right = mid - 1;
        if (nums[mid] == target) result = mid;
    }
    return result;
}
```

11.5 Practice Questions

Question 1: Find Peak Element

```
int findPeakElement(vector<int>& nums) {
    int left = 0, right = nums.size() - 1;

    while (left < right) {
        int mid = left + (right - left) / 2;
        if (nums[mid] > nums[mid + 1])
            right = mid;
        else
            left = mid + 1;
    }
    return left;
}
// Peak element is greater than its neighbors
```

Question 2: Search in 2D Matrix

```
bool searchMatrix(vector<vector<int>>& matrix, int target) {
    if (matrix.empty()) return false;

    int m = matrix.size(), n = matrix[0].size();
    int left = 0, right = m * n - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2;
        int midVal = matrix[mid / n][mid % n];

        if (midVal == target) return true;
        if (midVal < target) left = mid + 1;
        else right = mid - 1;
    }
    return false;
}
```

Interview Question: Why is binary search O(log n)?

Answer: Binary search halves the search space in each iteration. Starting with n elements, after k iterations we have n/(2^k) elements. We stop when n/(2^k) = 1, which means k = log n.

Chapter 12: Dynamic Programming

Dynamic Programming (DP) is a method for solving complex problems by breaking them down into simpler subproblems, solving each subproblem only once, and storing their solutions.

12.1 Key Concepts

1. Optimal Substructure

A problem has optimal substructure if an optimal solution can be constructed from optimal solutions of its subproblems.

2. Overlapping Subproblems

When a recursive algorithm revisits the same subproblems multiple times, we say the problem has overlapping subproblems.

12.2 Approaches to DP

1. Memoization (Top-Down)

Start from the main problem and recursively solve subproblems, storing results to avoid recomputation.

2. Tabulation (Bottom-Up)

Start from the smallest subproblems and build up to the main problem, filling a table iteratively.

12.3 Classic DP Problems

1. Fibonacci Numbers

```
// Memoization
int fibMemo(int n, vector<int>& dp) {
    if (n <= 1) return n;
    if (dp[n] != -1) return dp[n];
    return dp[n] = fibMemo(n-1, dp) + fibMemo(n-2, dp);
}

// Tabulation
int fibTab(int n) {
    if (n <= 1) return n;
    vector<int> dp(n + 1);
```

```
    dp[0] = 0; dp[1] = 1;
    for (int i = 2; i <= n; i++)
        dp[i] = dp[i-1] + dp[i-2];
    return dp[n];
}
// Time: O(n), Space: O(n) - can be optimized to O(1)
```

2. Climbing Stairs

You can climb 1 or 2 steps at a time. Count ways to reach the nth step.

```
int climbStairs(int n) {
    if (n <= 2) return n;
    int prev2 = 1, prev1 = 2;
    for (int i = 3; i <= n; i++) {
        int curr = prev1 + prev2;
        prev2 = prev1;
        prev1 = curr;
    }
    return prev1;
}
// Time: O(n), Space: O(1)
```

3. Longest Common Subsequence (LCS)

```
int lcs(string s1, string s2) {
    int m = s1.length(), n = s2.length();
    vector<vector<int>> dp(m + 1, vector<int>(n + 1, 0));

    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (s1[i-1] == s2[j-1])
                dp[i][j] = 1 + dp[i-1][j-1];
            else
                dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
        }
    }
    return dp[m][n];
}
// Time: O(m*n), Space: O(m*n)
```

4. 0/1 Knapsack Problem

Given weights and values of items, maximize value without exceeding weight capacity.

```
int knapsack(vector<int>& wt, vector<int>& val, int W, int n) {
    vector<vector<int>> dp(n + 1, vector<int>(W + 1, 0));
```

```
    for (int i = 1; i <= n; i++) {
        for (int w = 1; w <= W; w++) {
            if (wt[i-1] <= w)
                dp[i][w] = max(val[i-1] + dp[i-1][w-wt[i-1]], dp[i-1][w]);
            else
                dp[i][w] = dp[i-1][w];
        }
    }
    return dp[n][W];
}
// Time: O(n*W), Space: O(n*W)
```

5. Coin Change

```
int coinChange(vector<int>& coins, int amount) {
    vector<int> dp(amount + 1, amount + 1);
    dp[0] = 0;

    for (int i = 1; i <= amount; i++) {
        for (int coin : coins) {
            if (coin <= i)
                dp[i] = min(dp[i], 1 + dp[i - coin]);
        }
    }
    return dp[amount] > amount ? -1 : dp[amount];
}
// Minimum coins needed to make amount
```

6. Longest Increasing Subsequence

```
int lengthOfLIS(vector<int>& nums) {
    int n = nums.size();
    vector<int> dp(n, 1);

    for (int i = 1; i < n; i++) {
        for (int j = 0; j < i; j++) {
            if (nums[i] > nums[j])
                dp[i] = max(dp[i], dp[j] + 1);
        }
    }
    return *max_element(dp.begin(), dp.end());
}
// Time: O(n²), can be optimized to O(n log n)
```

7. Edit Distance

Minimum operations to convert one string to another (insert, delete, replace).

```
int minDistance(string word1, string word2) {
    int m = word1.length(), n = word2.length();
    vector<vector<int>> dp(m + 1, vector<int>(n + 1));

    for (int i = 0; i <= m; i++) dp[i][0] = i;
    for (int j = 0; j <= n; j++) dp[0][j] = j;

    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (word1[i-1] == word2[j-1])
                dp[i][j] = dp[i-1][j-1];
            else
                dp[i][j] = 1 + min({dp[i-1][j], dp[i][j-1], dp[i-1][j-1]});
        }
    }
    return dp[m][n];
}
// delete, insert, replace
```

12.4 Practice Questions

Question 1: House Robber

```
int rob(vector<int>& nums) {
    int n = nums.size();
    if (n == 0) return 0;
    if (n == 1) return nums[0];

    int prev2 = nums[0];
    int prev1 = max(nums[0], nums[1]);

    for (int i = 2; i < n; i++) {
        int curr = max(prev1, prev2 + nums[i]);
        prev2 = prev1;
        prev1 = curr;
    }
    return prev1;
}
// Can't rob adjacent houses
```

Question 2: Maximum Subarray (Kadane's Algorithm)

```
int maxSubArray(vector<int>& nums) {
    int maxSoFar = nums[0];
    int maxEndingHere = nums[0];

    for (int i = 1; i < nums.size(); i++) {
```

```
    maxEndingHere = max(nums[i], maxEndingHere + nums[i]);
    maxSoFar = max(maxSoFar, maxEndingHere);
  }
  return maxSoFar;
}
// Time: O(n), Space: O(1)
```

Interview Question: When to use DP?

Answer: Use DP when the problem has optimal substructure and overlapping subproblems. Look for problems involving optimization (max/min), counting, or checking existence/possibility.

Chapter 13: Greedy Algorithms

Greedy algorithms make the locally optimal choice at each step with the hope of finding a global optimum. They are simple and efficient but do not always produce the optimal solution.

13.1 Key Characteristics

- Greedy Choice Property: A global optimum can be reached by choosing local optima
- Optimal Substructure: An optimal solution contains optimal solutions to subproblems

13.2 Classic Greedy Problems

1. Activity Selection Problem

Select the maximum number of activities that don't overlap.

```
int activitySelection(vector<pair<int,int>>& activities) {
    // Sort by finish time
    sort(activities.begin(), activities.end(),
         [](auto& a, auto& b) { return a.second < b.second; });

    int count = 1;
    int lastFinish = activities[0].second;

    for (int i = 1; i < activities.size(); i++) {
        if (activities[i].first >= lastFinish) {
            count++;
            lastFinish = activities[i].second;
        }
    }
    return count;
}
// Time: O(n log n)
```

2. Fractional Knapsack

Unlike 0/1 knapsack, we can take fractions of items.

```
double fractionalKnapsack(vector<int>& wt, vector<int>& val, int W) {
    int n = wt.size();
    vector<pair<double, int>> ratio;  // (value/weight, index)

    for (int i = 0; i < n; i++)
        ratio.push_back({(double)val[i]/wt[i], i});
```

```
    sort(ratio.rbegin(), ratio.rend());

    double maxValue = 0;
    for (auto& r : ratio) {
        int idx = r.second;
        if (wt[idx] <= W) {
            maxValue += val[idx];
            W -= wt[idx];
        } else {
            maxValue += r.first * W;
            break;
        }
    }
    return maxValue;
}
// Time: O(n log n)
```

3. Minimum Number of Coins

```
vector<int> minCoins(vector<int>& coins, int amount) {
    sort(coins.rbegin(), coins.rend());
    vector<int> result;

    for (int coin : coins) {
        while (amount >= coin) {
            amount -= coin;
            result.push_back(coin);
        }
    }
    return result;
}
// Works for canonical coin systems like US coins
```

4. Huffman Coding

Creates an optimal prefix code for data compression based on character frequencies.

5. Job Sequencing with Deadlines

```
int jobSequencing(vector<Job>& jobs) {
    // Sort by profit descending
    sort(jobs.begin(), jobs.end(),
         [](Job& a, Job& b) { return a.profit > b.profit; });

    int maxDeadline = 0;
    for (auto& j : jobs) maxDeadline = max(maxDeadline, j.deadline);
```

```
    vector<int> slot(maxDeadline + 1, -1);
    int maxProfit = 0;

    for (auto& job : jobs) {
        for (int j = job.deadline; j > 0; j--) {
            if (slot[j] == -1) {
                slot[j] = job.id;
                maxProfit += job.profit;
                break;
            }
        }
    }
    return maxProfit;
}
```

13.3 Greedy vs Dynamic Programming

Aspect	Greedy	Dynamic Programming
Approach	Local optimal choice	Global optimal solution
Guarantee	May not be optimal	Always optimal
Speed	Faster	Slower
Examples	Dijkstra, Huffman	Knapsack, LCS

13.4 Practice Questions

Question 1: Jump Game

```
bool canJump(vector<int>& nums) {
    int maxReach = 0;
    for (int i = 0; i < nums.size(); i++) {
        if (i > maxReach) return false;
        maxReach = max(maxReach, i + nums[i]);
    }
    return true;
}
```

Question 2: Assign Cookies

```
int findContentChildren(vector<int>& g, vector<int>& s) {
    sort(g.begin(), g.end());
    sort(s.begin(), s.end());
```

```
    int child = 0, cookie = 0;
    while (child < g.size() && cookie < s.size()) {
        if (s[cookie] >= g[child])
            child++;
        cookie++;
    }
    return child;
}
// Satisfy maximum children with cookies
```

Interview Question: When does greedy fail?

Answer: Greedy fails when local optimal choices don't lead to global optimum. Example: 0/1 Knapsack cannot be solved greedily because we cannot take fractions of items - we need DP for optimal solution.

Chapter 14: Backtracking

Backtracking is a systematic way of trying out different sequences of decisions until a solution is found. When a path doesn't lead to a solution, we backtrack and try another path.

14.1 Key Concepts

- State Space Tree: Represents all possible states of the problem
- Pruning: Skip branches that cannot lead to a valid solution
- Base Case: Condition to stop recursion

14.2 Classic Backtracking Problems

1. N-Queens Problem

Place N queens on an NxN chessboard such that no two queens attack each other.

```
void solveNQueens(int n) {
    vector<string> board(n, string(n, '.'));
    solve(board, 0, n);
}

void solve(vector<string>& board, int row, int n) {
    if (row == n) {
        printBoard(board);
        return;
    }

    for (int col = 0; col < n; col++) {
        if (isValid(board, row, col, n)) {
            board[row][col] = 'Q';
            solve(board, row + 1, n);
            board[row][col] = '.';  // Backtrack
        }
    }
}

bool isValid(vector<string>& board, int row, int col, int n) {
    // Check column
    for (int i = 0; i < row; i++)
        if (board[i][col] == 'Q') return false;

    // Check diagonals
```

```
    for (int i = row-1, j = col-1; i >= 0 && j >= 0; i--, j--)
        if (board[i][j] == 'Q') return false;

    for (int i = row-1, j = col+1; i >= 0 && j < n; i--, j++)
        if (board[i][j] == 'Q') return false;

    return true;
}
```

2. Sudoku Solver

```
bool solveSudoku(vector<vector<char>>& board) {
    for (int i = 0; i < 9; i++) {
        for (int j = 0; j < 9; j++) {
            if (board[i][j] == '.') {
                for (char c = '1'; c <= '9'; c++) {
                    if (isValid(board, i, j, c)) {
                        board[i][j] = c;
                        if (solveSudoku(board)) return true;
                        board[i][j] = '.';  // Backtrack
                    }
                }
                return false;
            }
        }
    }
    return true;
}
```

3. Permutations

```
void permute(vector<int>& nums, int start, vector<vector<int>>& result) {
    if (start == nums.size()) {
        result.push_back(nums);
        return;
    }

    for (int i = start; i < nums.size(); i++) {
        swap(nums[start], nums[i]);
        permute(nums, start + 1, result);
        swap(nums[start], nums[i]);  // Backtrack
    }
}
// Time: O(n!), Space: O(n)
```

4. Subsets

```
void subsets(vector<int>& nums, int index, vector<int>& current,
```

```
        vector<vector<int>>& result) {
    result.push_back(current);

    for (int i = index; i < nums.size(); i++) {
        current.push_back(nums[i]);
        subsets(nums, i + 1, current, result);
        current.pop_back();  // Backtrack
    }
}
// Generate all subsets (power set)
```

5. Combination Sum

```
void combinationSum(vector<int>& candidates, int target, int start,
                    vector<int>& current, vector<vector<int>>& result) {
    if (target == 0) {
        result.push_back(current);
        return;
    }

    for (int i = start; i < candidates.size(); i++) {
        if (candidates[i] <= target) {
            current.push_back(candidates[i]);
            combinationSum(candidates, target - candidates[i], i, current, result);
            current.pop_back();  // Backtrack
        }
    }
}
```

6. Word Search

```
bool exist(vector<vector<char>>& board, string word) {
    for (int i = 0; i < board.size(); i++)
        for (int j = 0; j < board[0].size(); j++)
            if (dfs(board, word, 0, i, j))
                return true;
    return false;
}

bool dfs(vector<vector<char>>& board, string& word, int index, int i, int j) {
    if (index == word.length()) return true;
    if (i < 0 || j < 0 || i >= board.size() || j >= board[0].size())
        return false;
    if (board[i][j] != word[index]) return false;

    char temp = board[i][j];
    board[i][j] = '#';  // Mark as visited
```

```
    bool found = dfs(board, word, index + 1, i+1, j) ||
                 dfs(board, word, index + 1, i-1, j) ||
                 dfs(board, word, index + 1, i, j+1) ||
                 dfs(board, word, index + 1, i, j-1);

    board[i][j] = temp;  // Restore
    return found;
}
```

14.3 Practice Questions

Question 1: Palindrome Partitioning

```
void partition(string s, int start, vector<string>& current,
               vector<vector<string>>& result) {
    if (start == s.length()) {
        result.push_back(current);
        return;
    }

    for (int i = start; i < s.length(); i++) {
        if (isPalindrome(s, start, i)) {
            current.push_back(s.substr(start, i - start + 1));
            partition(s, i + 1, current, result);
            current.pop_back();
        }
    }
}
```

Question 2: Rat in a Maze

```
bool solveMaze(vector<vector<int>>& maze, int x, int y,
               vector<vector<int>>& sol) {
    int n = maze.size();

    if (x == n-1 && y == n-1 && maze[x][y] == 1) {
        sol[x][y] = 1;
        return true;
    }

    if (isSafe(maze, x, y)) {
        sol[x][y] = 1;

        if (solveMaze(maze, x+1, y, sol)) return true;
        if (solveMaze(maze, x, y+1, sol)) return true;

        sol[x][y] = 0;  // Backtrack
```

```
    }
    return false;
}
```

Interview Question: How to optimize backtracking?

Answer: Use pruning to eliminate invalid paths early, memoization to avoid recomputing states, and heuristics to explore promising paths first. Also consider iterative approaches or dynamic programming when applicable.

Chapter 15: Advanced Data Structures

This chapter covers advanced data structures that are essential for solving complex problems efficiently.

15.1 Tries (Prefix Trees)

A Trie is a tree-like data structure used to store a dynamic set of strings, where the keys are usually strings.

Trie Structure

```
struct TrieNode {
    TrieNode* children[26];
    bool isEndOfWord;

    TrieNode() {
        isEndOfWord = false;
        for (int i = 0; i < 26; i++)
            children[i] = NULL;
    }
};
```

Trie Operations

```
void insert(TrieNode* root, string key) {
    TrieNode* curr = root;
    for (char c : key) {
        int index = c - 'a';
        if (!curr->children[index])
            curr->children[index] = new TrieNode();
        curr = curr->children[index];
    }
    curr->isEndOfWord = true;
}

bool search(TrieNode* root, string key) {
    TrieNode* curr = root;
    for (char c : key) {
        int index = c - 'a';
        if (!curr->children[index])
            return false;
        curr = curr->children[index];
    }
    return curr->isEndOfWord;
```

```
}

bool startsWith(TrieNode* root, string prefix) {
    TrieNode* curr = root;
    for (char c : prefix) {
        int index = c - 'a';
        if (!curr->children[index])
            return false;
        curr = curr->children[index];
    }
    return true;
}
```

Applications of Trie

- Auto-complete and spell checking
- IP routing (Longest prefix matching)
- Pattern matching
- Word games and puzzles

15.2 Segment Trees

A Segment Tree is a tree data structure for storing intervals or segments. It allows querying which stored segments contain a given point.

Building a Segment Tree

```
void build(vector<int>& arr, int tree[], int node, int start, int end) {
    if (start == end) {
        tree[node] = arr[start];
    } else {
        int mid = (start + end) / 2;
        build(arr, tree, 2*node, start, mid);
        build(arr, tree, 2*node+1, mid+1, end);
        tree[node] = tree[2*node] + tree[2*node+1];
    }
}
// Query sum in range [l, r]
int query(int tree[], int node, int start, int end, int l, int r) {
    if (r < start || end < l) return 0;
    if (l <= start && end <= r) return tree[node];
    int mid = (start + end) / 2;
    return query(tree, 2*node, start, mid, l, r) +
           query(tree, 2*node+1, mid+1, end, l, r);
}
```

```
// Time: O(log n) for query and update
```

15.3 Binary Indexed Tree (Fenwick Tree)

Fenwick Tree is a data structure that can efficiently update elements and calculate prefix sums.

```
class FenwickTree {
    vector<int> bit;
    int n;

public:
    FenwickTree(int size) {
        n = size;
        bit.resize(n + 1, 0);
    }

    void update(int index, int delta) {
        for (; index <= n; index += index & -index)
            bit[index] += delta;
    }

    int query(int index) {  // Prefix sum [1..index]
        int sum = 0;
        for (; index > 0; index -= index & -index)
            sum += bit[index];
        return sum;
    }

    int rangeQuery(int l, int r) {
        return query(r) - query(l - 1);
    }
};
// Time: O(log n)
```

15.4 Disjoint Set Union (Union-Find)

Union-Find is a data structure that keeps track of a partition of a set into disjoint subsets.

```
class UnionFind {
    vector<int> parent, rank_;

public:
    UnionFind(int n) {
        parent.resize(n);
        rank_.resize(n, 0);
        for (int i = 0; i < n; i++)
            parent[i] = i;
```

```
    }

    int find(int x) {
        if (parent[x] != x)
            parent[x] = find(parent[x]);  // Path compression
        return parent[x];
    }

    void unite(int x, int y) {
        int px = find(x), py = find(y);
        if (px == py) return;

        // Union by rank
        if (rank_[px] < rank_[py]) swap(px, py);
        parent[py] = px;
        if (rank_[px] == rank_[py]) rank_[px]++;
    }

    bool connected(int x, int y) {
        return find(x) == find(y);
    }
};
// Amortized Time: O(alpha(n)) ~= O(1)
```

15.5 LRU Cache

LRU (Least Recently Used) Cache is a cache replacement policy that removes the least recently used items first.

```
class LRUCache {
    int capacity;
    list<pair<int, int>> cache;  // {key, value}
    unordered_map<int, list<pair<int,int>>::iterator> map;

public:
    LRUCache(int cap) : capacity(cap) {}

    int get(int key) {
        if (map.find(key) == map.end()) return -1;

        // Move to front
        cache.splice(cache.begin(), cache, map[key]);
        return map[key]->second;
    }

    void put(int key, int value) {
        if (map.find(key) != map.end()) {
```

```
        map[key]->second = value;
        cache.splice(cache.begin(), cache, map[key]);
        return;
    }

    if (cache.size() == capacity) {
        int lru = cache.back().first;
        cache.pop_back();
        map.erase(lru);
    }

    cache.push_front({key, value});
    map[key] = cache.begin();
  }
};
// Time: O(1) for both get and put
```

15.6 Practice Questions

Question 1: Implement Trie for Auto-complete

```
void autoComplete(TrieNode* root, string prefix) {
  TrieNode* curr = root;
  for (char c : prefix) {
    int index = c - 'a';
    if (!curr->children[index]) return;
    curr = curr->children[index];
  }
  printAllWords(curr, prefix);
}

void printAllWords(TrieNode* node, string prefix) {
  if (node->isEndOfWord) cout << prefix << endl;
  for (int i = 0; i < 26; i++) {
    if (node->children[i]) {
      char c = 'a' + i;
      printAllWords(node->children[i], prefix + c);
    }
  }
}
```

Question 2: Number of Provinces (Union-Find)

```
int findCircleNum(vector<vector<int>>& isConnected) {
  int n = isConnected.size();
  UnionFind uf(n);
```

```
    for (int i = 0; i < n; i++)
        for (int j = i + 1; j < n; j++)
            if (isConnected[i][j])
                uf.unite(i, j);

    unordered_set<int> provinces;
    for (int i = 0; i < n; i++)
        provinces.insert(uf.find(i));

    return provinces.size();
}
```

Interview Question: When to use Segment Tree vs Fenwick Tree?

Answer: Use Fenwick Tree for simpler range queries and point updates (simpler code, less memory). Use Segment Tree for more complex operations like range updates, range minimum/maximum queries, or when you need lazy propagation.

Chapter 16: Interview Preparation

This chapter contains commonly asked interview questions and their solutions. Practice these thoroughly to ace your technical interviews.

16.1 Arrays and Strings

Q1: Two Sum

Given an array and a target, find two numbers that add up to the target.

```
vector<int> twoSum(vector<int>& nums, int target) {
    unordered_map<int, int> map;
    for (int i = 0; i < nums.size(); i++) {
        int complement = target - nums[i];
        if (map.count(complement))
            return {map[complement], i};
        map[nums[i]] = i;
    }
    return {};
}
// Time: O(n), Space: O(n)
```

Q2: Best Time to Buy and Sell Stock

```
int maxProfit(vector<int>& prices) {
    int minPrice = INT_MAX;
    int maxProfit = 0;
    for (int price : prices) {
        minPrice = min(minPrice, price);
        maxProfit = max(maxProfit, price - minPrice);
    }
    return maxProfit;
}
// Time: O(n), Space: O(1)
```

Q3: Product of Array Except Self

```
vector<int> productExceptSelf(vector<int>& nums) {
    int n = nums.size();
    vector<int> result(n, 1);

    // Left products
    for (int i = 1; i < n; i++)
        result[i] = result[i-1] * nums[i-1];
```

```
    // Right products
    int right = 1;
    for (int i = n-1; i >= 0; i--) {
        result[i] *= right;
        right *= nums[i];
    }
    return result;
}
// Time: O(n), Space: O(1) excluding output
```

16.2 Linked Lists

Q4: Reverse Linked List

```
ListNode* reverseList(ListNode* head) {
    ListNode* prev = NULL;
    ListNode* curr = head;
    while (curr) {
        ListNode* next = curr->next;
        curr->next = prev;
        prev = curr;
        curr = next;
    }
    return prev;
}
// Time: O(n), Space: O(1)
```

Q5: Detect Cycle in Linked List

```
bool hasCycle(ListNode* head) {
    if (!head) return false;
    ListNode* slow = head, *fast = head;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) return true;
    }
    return false;
}
// Floyd's Cycle Detection - Time: O(n), Space: O(1)
```

16.3 Trees and Graphs

Q6: Maximum Depth of Binary Tree

```
int maxDepth(TreeNode* root) {
```

```
    if (!root) return 0;
    return 1 + max(maxDepth(root->left), maxDepth(root->right));
}
// Time: O(n), Space: O(h) where h is height
```

Q7: Number of Islands

```
int numIslands(vector<vector<char>>& grid) {
    if (grid.empty()) return 0;
    int count = 0;
    for (int i = 0; i < grid.size(); i++)
        for (int j = 0; j < grid[0].size(); j++)
            if (grid[i][j] == '1') {
                dfs(grid, i, j);
                count++;
            }
    return count;
}

void dfs(vector<vector<char>>& grid, int i, int j) {
    if (i < 0 || j < 0 || i >= grid.size() || j >= grid[0].size())
        return;
    if (grid[i][j] != '1') return;
    grid[i][j] = '0';
    dfs(grid, i+1, j);
    dfs(grid, i-1, j);
    dfs(grid, i, j+1);
    dfs(grid, i, j-1);
}
```

16.4 Dynamic Programming

Q8: Climbing Stairs

```
int climbStairs(int n) {
    if (n <= 2) return n;
    int prev2 = 1, prev1 = 2;
    for (int i = 3; i <= n; i++) {
        int curr = prev1 + prev2;
        prev2 = prev1;
        prev1 = curr;
    }
    return prev1;
}
// Fibonacci pattern - Time: O(n), Space: O(1)
```

Q9: Longest Palindromic Substring

```
string longestPalindrome(string s) {
    int n = s.length();
    if (n < 2) return s;

    int start = 0, maxLen = 1;
    for (int i = 0; i < n; i++) {
        // Odd length
        int l = i, r = i;
        while (l >= 0 && r < n && s[l] == s[r]) {
            if (r - l + 1 > maxLen) {
                start = l;
                maxLen = r - l + 1;
            }
            l--; r++;
        }
        // Even length
        l = i; r = i + 1;
        while (l >= 0 && r < n && s[l] == s[r]) {
            if (r - l + 1 > maxLen) {
                start = l;
                maxLen = r - l + 1;
            }
            l--; r++;
        }
    }
    return s.substr(start, maxLen);
}
// Expand around center - Time: O(n²), Space: O(1)
```

16.5 System Design Concepts

Key Concepts to Know

- Time and Space Complexity Analysis
- Trade-offs between different data structures
- When to use specific algorithms
- Scalability considerations
- Caching strategies
- Database indexing

16.6 Tips for Technical Interviews

Before the Interview

- Practice coding on whiteboard or paper
- Review time complexities of common operations
- Understand the problem before jumping to solution
- Practice explaining your thought process

During the Interview

- Clarify the problem and constraints
- Start with a brute force solution
- Optimize step by step
- Test your solution with examples
- Discuss time and space complexity

Common Mistakes to Avoid

- Not asking clarifying questions
- Jumping into code without planning
- Ignoring edge cases
- Not testing the solution
- Poor variable naming

16.7 Quick Reference: Time Complexities

Data Structure	Access	Search
Array	O(1)	O(n)
Linked List	O(n)	O(n)
Binary Search Tree	O(log n)	O(log n)
Hash Table	N/A	O(1)
Heap	O(1) max/min	O(n)

Final Words

Remember that mastering Data Structures and Algorithms is a journey, not a destination. Keep practicing, stay curious, and never stop learning. The concepts in this book form the foundation of computer science and will serve you throughout your career.

Good luck with your interviews and happy coding!

- Mohammad Faiz

Data Structures & Algorithms

The Complete A to Z Guide

Mohammad Faiz

© 2026 All Rights Reserved